V2V-SIH

Decentralized Vehicle-to-Vehicle Communication & Edge Intelligence

A browser-based prototype in which one tab is the entire V2V world. Vehicles exchange real-time safety messages over a simulated wireless channel, each car detects collision risk locally, and a roadside edge node observes the traffic — with no cloud anywhere in the safety-critical loop.

Built with React + TypeScript + HTML Canvas (Vite). No backend.

Project & Code Documentation

How the project is built, how the code flows, and what every number on the screen means — phases 1 to 5.

1. What this project is

The problem: cars need to warn each other about danger **instantly** — a sudden hard brake, a stopped vehicle ahead — and they cannot afford to wait for a request to travel to a distant cloud server and back. This prototype demonstrates a **decentralized** alternative: every vehicle broadcasts what it is doing, every vehicle listens and decides for itself, and safety never depends on a central computer.

The whole simulation runs inside a single web page. There is no real radio and no server — instead an explicit software **channel model** reproduces the things that make wireless hard (range limits, delay, jitter, packet loss). Because those effects are modelled honestly, every number the dashboard shows is a **real measurement** of the simulation, not a decorative figure typed into the UI.

> **The one idea everything serves**
> A vehicle only ever acts on **its own state** and **the messages it has personally received**. No vehicle is allowed to read the global list of all cars. That single rule is what makes the system genuinely decentralized rather than centralized code wearing a decentralized diagram.

The technology, in one line

- **React + TypeScript** for the interface and dashboard panels.
- **HTML Canvas** for drawing the road and vehicles.
- **Plain TypeScript classes** for the simulation (physics, channel, protocol, risk) — no game engine, no libraries.
- **Vite** as the dev server / build tool.

2. How the code is organised

The code is split by responsibility. Each folder does one job, and the dependencies flow one way: the simulation knows nothing about pixels or React; the UI knows nothing about metres.

Folder / file	Responsibility
sim/config.ts	All tunable constants in one place — road size, vehicle size, timestep, channel and risk thresholds. The single source of truth so numbers are never scattered through the code.
sim/rng.ts	A seeded pseudo-random generator (mulberry32). The same seed always produces the same run. Real randomness (Math.random) is never used inside the simulation.
sim/vehicle.ts	One car: its own position, speed, inbox, world model, and risk state. It integrates its own physics and decides its own braking — nothing more.
sim/world.ts	Owns all the vehicles and runs one fixed simulation step: physics, beacons, delivery, protocol, risk, edge observation.
sim/message.ts	The V2V message format (a CAM 'here I am' beacon or an emergency-braking message): sender, sequence number, position, speed, TTL.

Folder / file	Responsibility
sim/metrics.ts	Measures the channel: latency percentiles, packet reception ratio, loss, relay and duplicate counts — over a rolling 3-second window.
channel/channel.ts	The simulated radio: range check, distance-based packet loss, delay + jitter, and scheduled delivery into each receiver's inbox.
protocol/protocol.ts	The decentralized rules: reject duplicate messages, learn from new ones, and relay them onward until their TTL runs out.
risk/risk.ts	The collision-risk engine: from a car's own world model, compute time-to-collision (TTC) and a safe-distance check, returning SAFE / WARNING / DANGER.
edge/rsu.ts	The roadside unit — a parallel observer that aggregates connected count, density, and congestion. Never feeds back into any car's decision.
render/canvasRenderer.ts	The only place metres are converted to pixels. Draws the road, cars, links, range rings, the RSU and the HUD.
App.tsx / ui/*	The React shell: the animation loop, the controls, and the dashboard panels that sample the simulation 10 times a second.

3. How the simulation actually runs

This is the heart of 'how the code works'. Two clocks run at the same time, on purpose:

- **The physics clock** — the simulation advances in fixed **50 ms** steps (20 times a second). This never changes, no matter how fast or slow your screen refreshes, so every measurement means the same thing every run.
- **The drawing clock** — the canvas redraws every animation frame (whatever your monitor does), and the React dashboard refreshes only **10 times a second**. Updating React on every physics step would overwhelm it.

A real-time accumulator bridges them: however much wall-clock time has passed, the world runs exactly the right number of 50 ms steps to catch up. Each of those steps runs the same seven-stage pipeline:

```
ONE SIMULATION STEP  (World.step, every 50 ms)
-----------------------------------------------------------------
 1. PHYSICS      each car moves itself: v += a*dt, then x += v*dt
                    |
 2. BEACON       every ~250 ms each car broadcasts a CAM message
                 ('here I am: position, speed, heading') into ...
                    |
 3. CHANNEL      for each other car: in range? -> roll packet loss
                 -> schedule delivery after delay + jitter
                    |
 4. DELIVER      messages whose delay has elapsed land in inboxes
                    |
 5. PROTOCOL     each car reads its OWN inbox:
                    dedupe (seen before? drop)
                    learn  (update my world model)
                    relay  (ttl>1? re-broadcast, ttl-1)
                    |
 6. RISK         each car checks its world model for the closest
                 car ahead -> TTC -> SAFE / WARNING / DANGER
                 -> sets its own braking
                    |
 7. OBSERVE      the RSU aggregates the road (density, congestion)
                 -- purely watching, never steering anyone
-----------------------------------------------------------------
```

Notice that stages 5 and 6 only ever touch **one car's own inbox and state**. That is the decentralization rule, enforced by the shape of the code: those functions are never even given the list of all cars.

4. The five build phases

The prototype was built in five layers, each adding one capability and each with a single test that proves it works. Every layer keeps the runs reproducible (same seed = same run), which is what makes the measurements trustworthy.

Phase 1 · Road & Motion

The visible skeleton: a three-lane road and a dozen cars moving under a fixed-timestep physics loop, with Start / Pause / Reset controls. No communication yet — just movement done correctly.

What it adds

- A **World** that owns the vehicles and steps them at a fixed 50 ms.
- A **seeded random generator** so spawns are reproducible from day one.
- A **canvas renderer** that draws the road and cars, converting metres to pixels in exactly one place.
- The animation loop with the real-time accumulator described in section 3.

How it works

```
spawn cars (seeded)  ->  every 50 ms: each car integrates its
own position  ->  canvas redraws from the world state
```

Done when

Cars move smoothly regardless of screen or tab focus, and pressing Reset reproduces the identical run twice (same seed = pixel-identical positions at every tick).

Phase 2 · Channel & Metrics

Turns the numbers honest. A message is no longer a plain function call — it passes through an explicit radio model with range, delay, jitter and distance-based loss, so latency and reliability become real measurements.

What it adds

- A **Channel** that drops out-of-range messages, rolls packet loss that grows with distance, and delivers survivors after a delay.
- A **Metrics** recorder computing latency p50/p95, packet reception ratio (PRR) and loss over a rolling window.
- A live dashboard, a channel-range slider, and click-to-select + drag a car.

How a message travels

```
sender broadcasts  ->  for each other car:
   distance > range?          -> not reachable (ignored)
   random < loss(distance)?   -> dropped   (counts against PRR)
   otherwise                  -> delivered after delay + jitter
```

Done when

You drag a car toward the edge of its range ring and watch its link reliability (PRR) fall as distance-based loss climbs — then read OUT OF RANGE once it crosses the ring.

Phase 3 · Protocol & Decentralization

Makes 'decentralized' true. Until now messages arrived but nobody acted on them. Now each car runs the real protocol on its own inbox: reject duplicates, learn, and relay onward — building its own private picture of the road.

What it adds

- A **dedupe set** keyed by (sender, sequence number) so the same message is never processed twice.
- A per-car **world model** — what this car believes about others, built only from received messages.
- **Multi-hop relay**: a message with time-to-live (TTL) left is re-broadcast (TTL-1), so news reaches cars beyond direct range.

What each car does with a message

```
message arrives  ->  seen this (sender:seq) before?
   yes -> DROP  (duplicate rejected -- stops endless loops)
   no  -> record it in my world model  (note hop distance)
       -> ttl > 1 ?  re-broadcast a copy with ttl-1  (relay)
```

Done when

A message from a far car reaches you through a relay with its TTL correctly decremented, and duplicate copies are rejected. In the UI, a selected car shows green links (heard directly) and amber links (heard via relay).

Phase 4 · Risk & the Hero Scenario

The payoff. A collision-risk engine runs inside every car over the world model built in Phase 3, colours cars by danger, and the red Trigger Emergency button produces the whole demo: brake, broadcast, danger, propagate.

What it adds

- A **risk engine** computing time-to-collision (TTC) plus an RSS-lite safe-distance check for the closest car ahead.
- Three risk levels driving colour and braking: **SAFE** (green), **WARNING** (amber, ease off), **DANGER** (red, brake).
- The **Trigger Emergency** button and the headline **Warning time** metric.

The risk calculation (inside each car)

```
for the closest car AHEAD in my world model:
   vr = closing speed (how fast the gap is shrinking)
   vr <= 0 ?  -> we are separating -> SAFE  (important guard)
   TTC = distance / vr
        TTC > 5s   -> SAFE      TTC 3-5s -> WARNING
        TTC < 3s   -> DANGER  (brake hard)
```

Done when

One button press produces the full chain: the front car brakes and broadcasts it; the car behind computes its own TTC and hits DANGER; the wave of WARNING/DANGER then propagates backward down the column of cars.

Phase 5 · Edge Node

Adds a roadside unit (RSU) beside the road that taps the same channel and reports on traffic — connected count, density, congestion — without ever being part of a braking decision.

What it adds

- An **RSU** that aggregates the corridor into connected count, density (LOW/MED/HIGH) and a congestion flag.
- **Incident detection**: congestion is raised when the road is crawling on average, or when a car has stopped in coverage.

- A drawn mast on the road that turns red when the RSU detects congestion.

How it fits the architecture

The RSU is a **parallel observer**. It listens to the same channel and builds an aggregate picture, but nothing it computes ever flows back into a car's braking decision. It sits beside the safety loop, never inside it — which is exactly what keeps the system decentralized. Trigger the emergency, a car stops, and the RSU independently raises CONGESTION, while the braking itself stayed entirely on the vehicles.

> **Done when**
>
> The edge node observes the road live and independently flags an incident (congestion) when a car stops — while every safety decision stays local at the vehicle.

5. Reading the website — every number explained

This section is a plain-language reference for everything shown on screen. It is grouped by where the item appears: the top bar and controls, the road canvas itself, and each dashboard panel down the right-hand side.

5.1 Top bar & controls

Control / label	What it means
RUNNING / PAUSED	Status pill in the top-right. Whether the simulation clock is currently advancing.
Start / Pause	Runs or freezes the simulation. Physics only advances while running.
Reset	Rewinds to time zero with the same seed — producing an identical run. Clears any emergency.
Trigger Emergency	Makes one car (the front car, or the one you have selected) brake hard and broadcast an emergency message. Press again to clear it and let traffic recover.
Vehicles	Slider for how many cars are on the road (2–30). Changing it rebuilds the world.
Seed	The random seed. The same seed always reproduces the same run; change it for a different traffic pattern.

5.2 On the road (the canvas)

What you see	What it means
Cyan car	A vehicle in the SAFE state — no collision threat ahead.
Amber car	WARNING state — a car ahead is within 3–5 seconds' time-to-collision; it eases off.
Red car	DANGER (or the emergency braker) — time-to-collision under 3 seconds; it brakes hard.
V1 ... V12 label	Each car's identifier, shown above it.
TTC 2.3s label	Appears under a WARNING or DANGER car: its current time-to-collision with the car ahead.
White halo	The car you have selected (by clicking it).
Dashed ring	The selected car's communication range. Cars outside it cannot be heard directly.
Green link line	The selected car hears that neighbour DIRECTLY (zero relays).
Amber link line	The selected car hears that neighbour via a RELAY (multi-hop). Amber links reaching outside the ring are the visible proof of multi-hop knowledge.
Purple mast	The roadside unit (RSU) and its coverage band. Turns red and reads 'RSU · CONGESTION' when it detects an incident.
Red expanding rings	A sonar-style pulse from the emergency braker — a visual for the warning spreading outward.

What you see	What it means
HUD (bottom-left)	t = seconds elapsed, tick = number of 50 ms steps, vehicles = car count.

5.3 Warning time panel

The headline safety number.

Item	What it means
Warning time	Milliseconds from the moment the emergency braker starts decelerating to the moment the first car behind it reaches DANGER. Lower is better — it is how quickly the warning propagated. Reads a dash until an emergency is triggered, then 'measuring...', then the latched value.

5.4 System status panel (the channel)

Metric	What it means
Latency p50	Median message delay: half of delivered messages arrived faster than this. Typically ~11 ms.
Latency p95	95th-percentile delay: only 5% of messages were slower. The tail matters most for safety. Typically ~14 ms.
PRR	Packet Reception Ratio = delivered / in-range attempts. The fraction of messages that got through to cars that were in range. Higher is better.
Packet loss	Simply 1 - PRR. The fraction lost. Judges often ask for it by name.
... delivered / msg/s / window	The raw counts behind PRR, the delivered-message throughput per second, and the length of the rolling window (3 s) over which everything above is measured, so the numbers stay live.

5.5 Protocol panel (decentralization)

Metric	What it means
Relays /s	Multi-hop re-broadcasts per second — messages forwarded onward so distant cars hear them.
Max hops	The deepest relay chain seen recently (e.g. 2 = a message reached someone two hops from its origin).
Dupes rejected /s	Duplicate messages thrown away per second by the dedupe rule. A high number here is good — it is what stops relayed messages from looping forever.
Dedupe	Reads ACTIVE whenever duplicates are being rejected — confirmation the anti-loop protection is working.

5.6 Channel model panel (the knobs)

These are the *inputs* to the radio model, not measurements.

Setting	What it means
Range	Maximum communication distance (slider, live). Beyond it, messages are not delivered at all.
Base delay	The minimum delay every message pays (8 ms).
Jitter	Random extra delay added on top (0–6 ms) — what creates the spread between p50 and p95.
Loss @ 0 m	Packet-loss probability right next to the sender (1%).
Loss @ range	Packet-loss probability at the very edge of range (35%). Loss is interpolated by distance between these two.

5.7 Selected link panel (one car's view)

Appears when you click a car; shows that car's own risk and communication picture.

Item	What it means
SAFE / WARNING / DANGER badge	The selected car's current risk level.
TTC	Time-to-collision with the closest car ahead, in seconds. Shows the infinity symbol when nothing ahead is closing.
Safe dist (RSS)	The RSS-lite safe following distance for this car's speed: reaction distance plus braking distance. If the real gap is smaller, it flags INSIDE safe distance.
gap to threat	Actual distance to the car ahead being tracked, and whether that is inside or outside the safe distance.
Link: in range	How many cars are currently within this car's range (or OUT OF RANGE if isolated).
Link PRR / Link p50	Reception reliability and median latency, for messages received specifically by this one car.
World-model chips	The list of cars this one currently knows about, tagged by hop distance (green = direct, amber = relayed). This is the car's private, decentralized picture of the road.

5.8 Edge node panel (the RSU)

Item	What it means
Connected	How many vehicles are currently within the RSU's range.
Density	Traffic density over the observed span, categorised LOW / MED / HIGH (with the vehicles-per-km figure).
Avg speed	Mean speed of the connected vehicles.
Congestion	YES when the corridor is crawling on average, or when a car has stopped in coverage (an incident). Otherwise 'no'. The RSU raises this on its own, without affecting any car's driving.

6. Why it is built this way

Five design choices carry through every phase. They are not stylistic — each one protects the credibility of the demo.

- **Fixed 50 ms timestep, separate from drawing.** Physics runs at the same rate on any machine, so time-to-collision, latency and warning time mean the same thing every run.
- **Everything in metres and seconds inside the simulation.** Only the renderer converts to pixels, through one constant. Mixing units would silently corrupt every distance calculation.
- **A seeded random generator, never real randomness.** The same seed reproduces the same run — so a result can be shown twice and trusted, and different runs can be compared fairly.
- **Simulation state lives outside React.** The world is a plain class; React only samples it 10 times a second for the panels. This keeps 20 cars at 20 Hz smooth.
- **A vehicle only reads its own inbox and state.** Enforced by the code's shape — the protocol and risk functions are never handed the global car list, so the system cannot quietly become centralized.

Running it

```
npm install
npm run dev
# then open http://localhost:5173
```

A calm start shows a dozen green cars beaconing. Select a car to inspect its links and risk; drag it to the edge of range to watch reliability fall; press Trigger Emergency to see the danger wave roll backward and the RSU flag congestion.

V2V-SIH — decentralized V2V + edge intelligence. Prototype documentation, phases 1–5.